

Herança de Classes em Python

Arquitetura, Reuso de Código e a Estrutura de Objetos



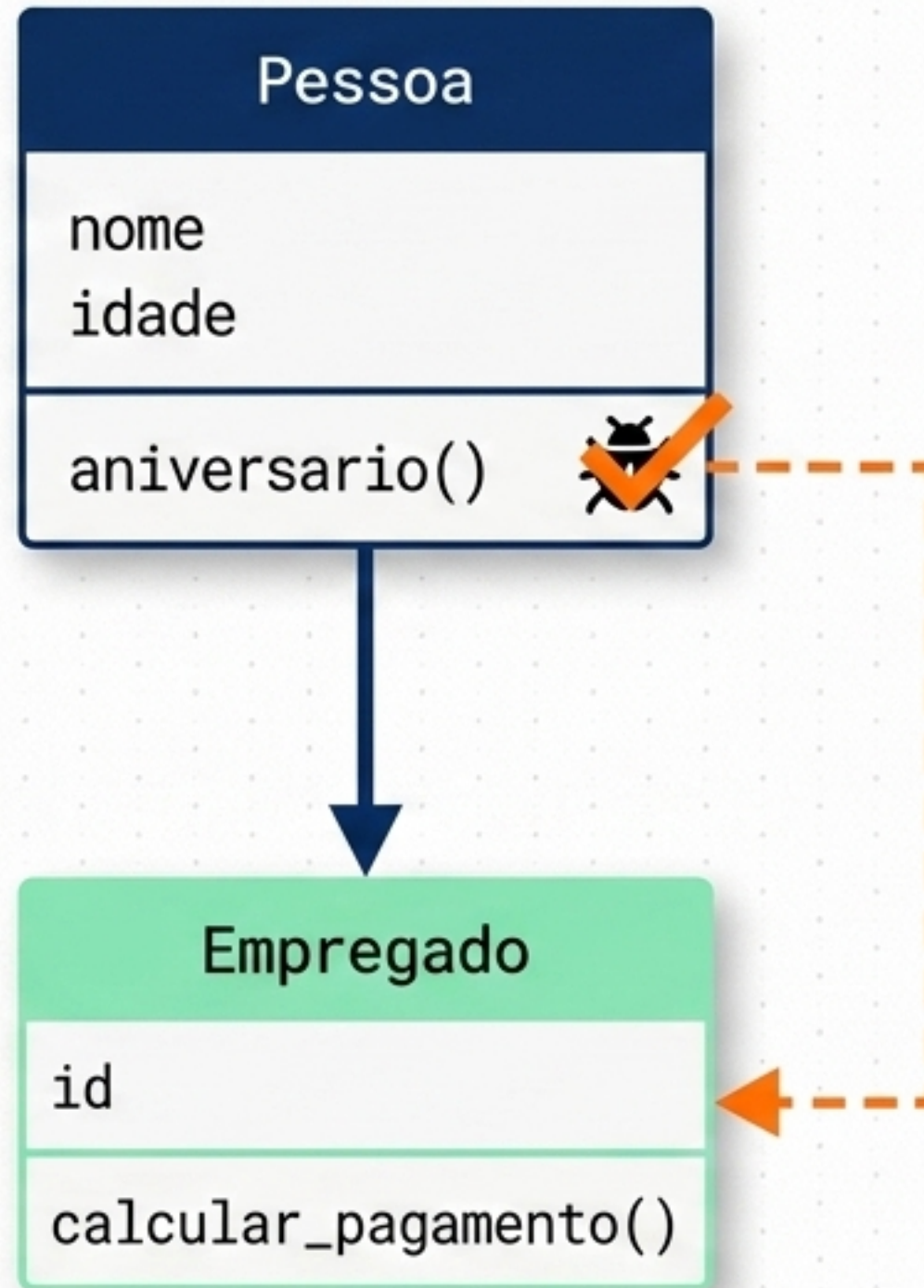
Sumário da Arquitetura

1	0 Conceito de Herança (0 Princípio DRY)
2	Terminologia e Hierarquia Oficial
3	Matriz de Diagnóstico: 'É-um' vs 'Faz-parte-de'
4	A Classe Invisível: object
5	Anatomia de uma Subclasse (Critérios de Uso)
6	Sobrescrita vs. Extensão (super())
7	Encapsulamento e Name Mangling
8	Herança Múltipla e o Problema do Diamante
9	Resumo (Cheat Sheet)

O Problema e a Solução: Por que Herança?

Código definido uma vez,
reutilizado sempre.

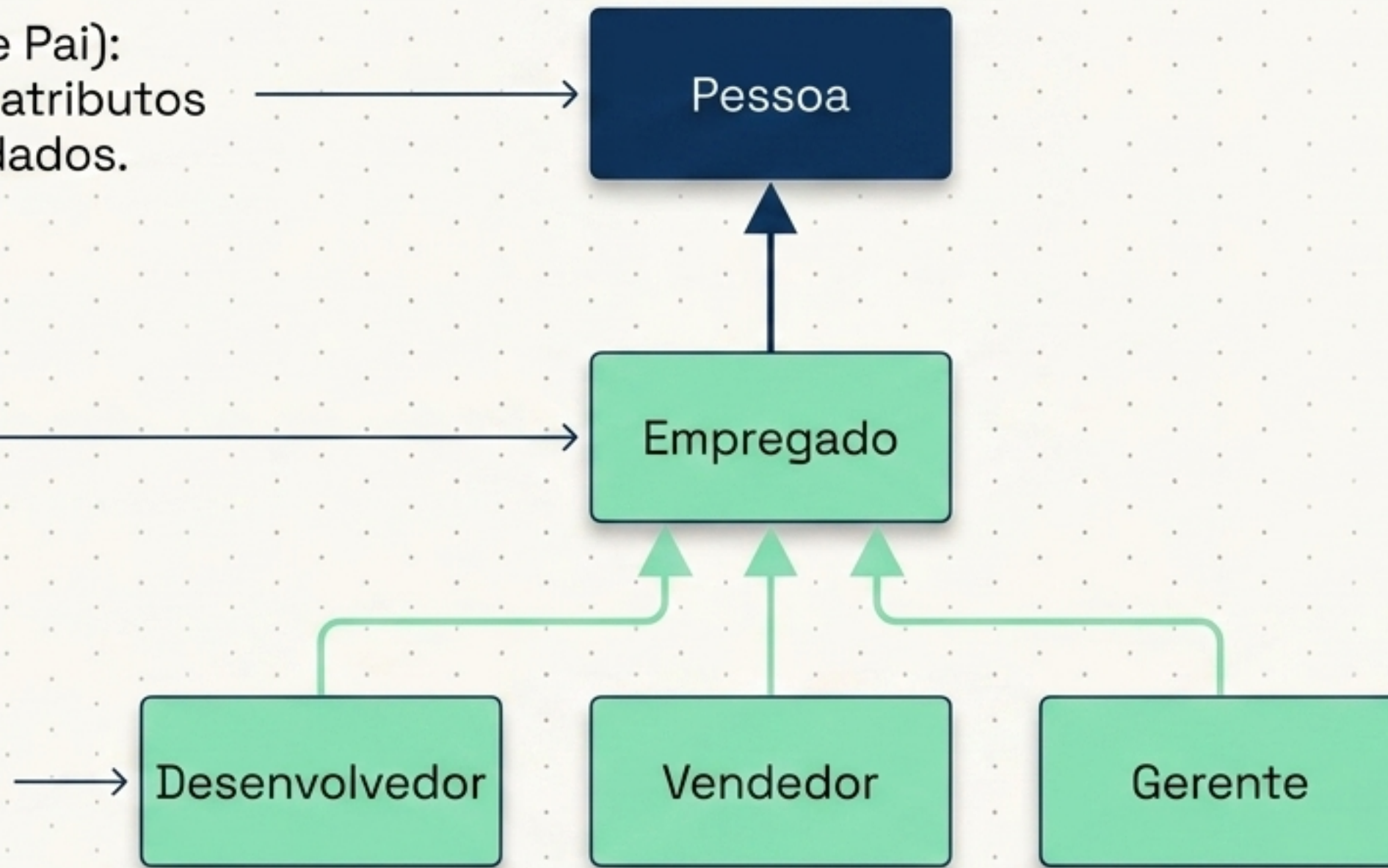
A correção de um bug na
superclasse é propagada
automaticamente para as
subclasses.



Terminologia Oficial da Orientação a Objetos

Superclasse (Classe Pai):
A classe da qual os atributos
e métodos são herdados.

Subclasse (Classe
Filha): Herda a
estrutura da
superclasse, mas é
uma classe
independente.



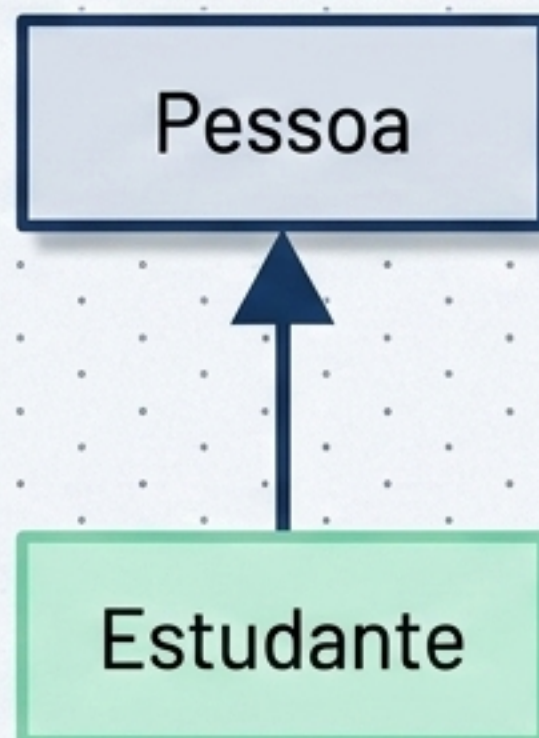
Hierarquia de
Herança

Matriz de Diagnóstico: “É-um” vs “Faz-parte-de”

Duas hierarquias estruturais frequentemente confundidas.

Herança / Subclassing

Relacionamento “É-um” (Is-a)



Implementado via sub-classes.
Um estudante **é uma** pessoa.

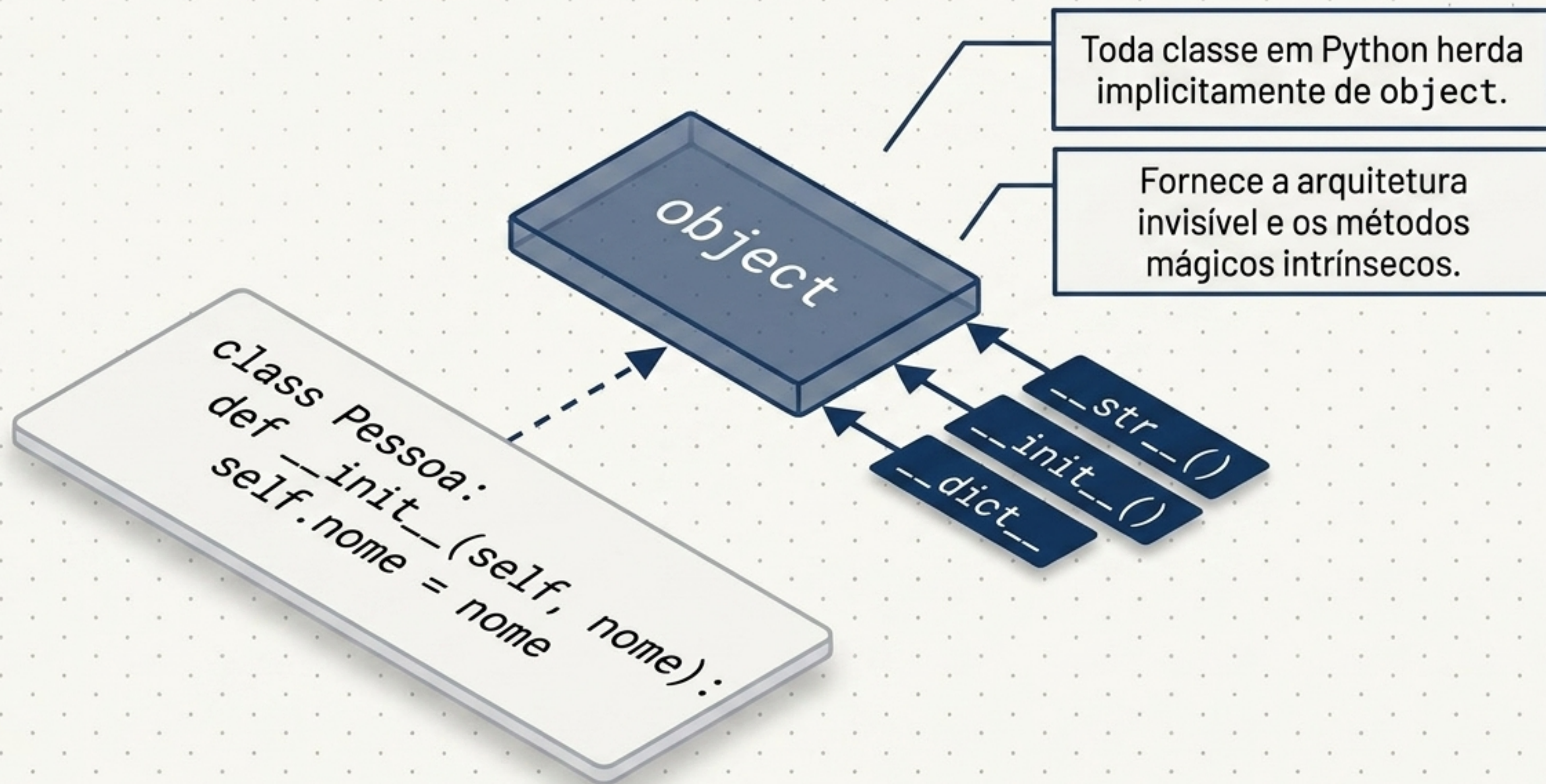
Composição / Instantiation

Relacionamento “Faz-parte-de” (Part-of)



Implementado via atributos de instância.
Um motor **faz parte de** um carro.
(Um motor não **é um** carro).

A Raiz de Tudo: A Classe object

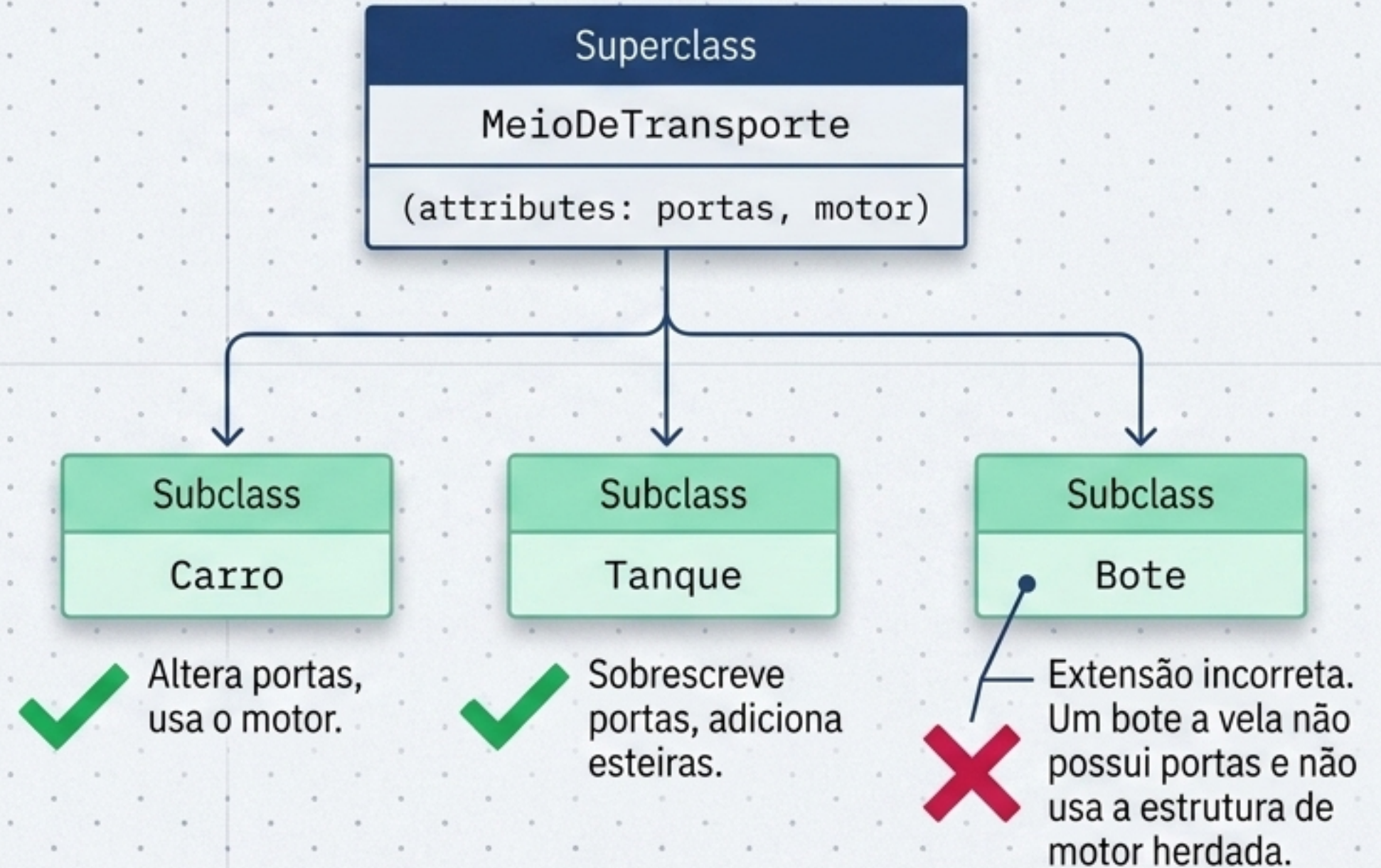


O Propósito de uma Subclasse: Quando usar?

Critérios de Herança Válida

1. Altera a interface externa.
2. Altera a interface externa.
2. Altera a implementação interna.
3. Adiciona comportamento referenciando o pai.

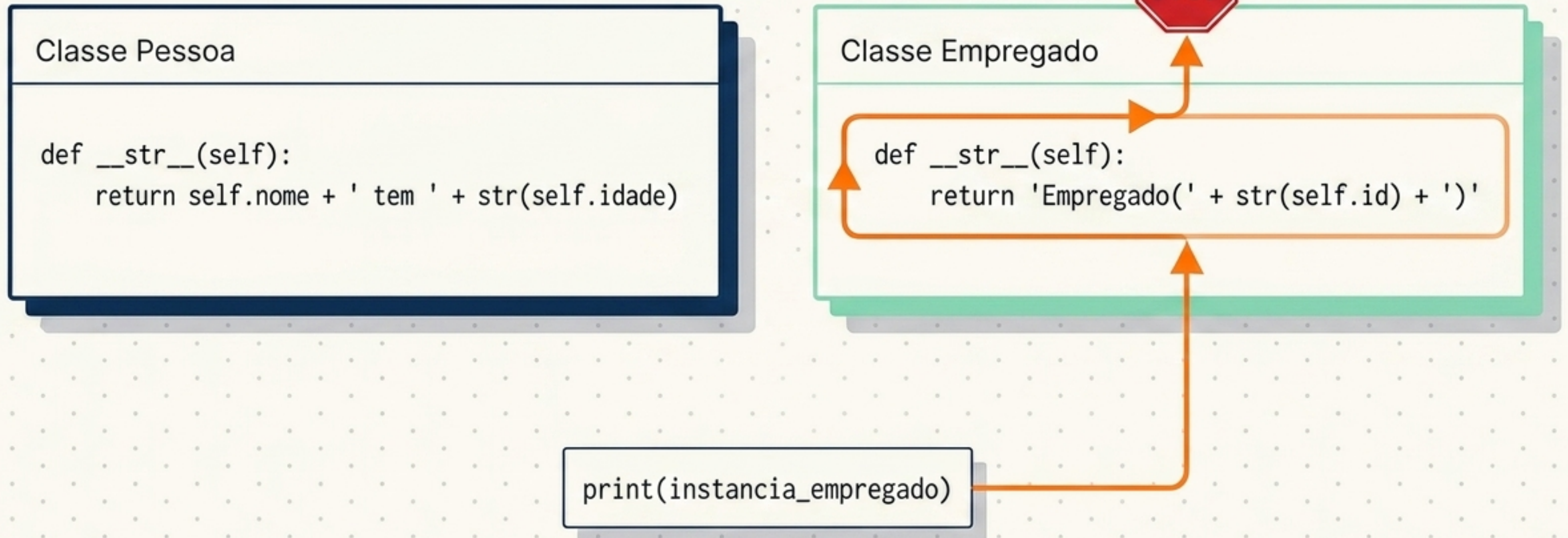
(Se não atender a nenhum critério, não use herança).



Sobrescrita de Métodos (Overriding)

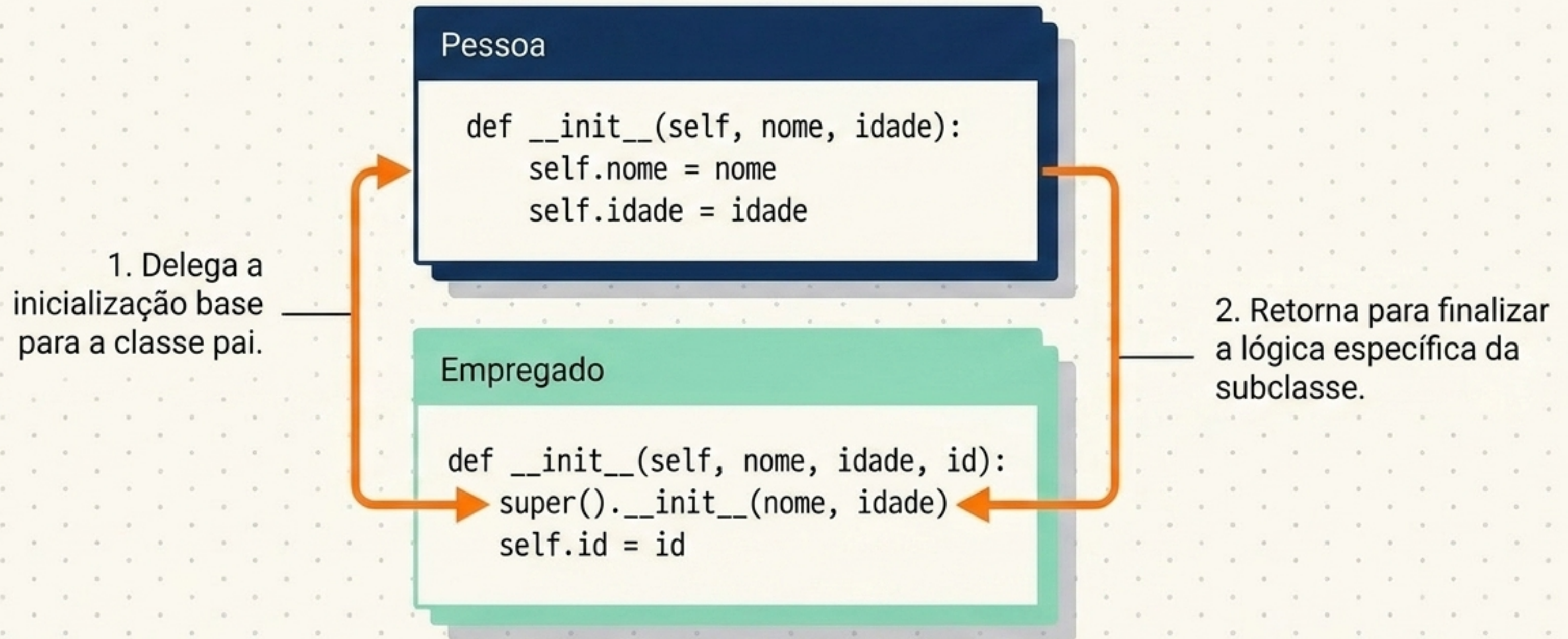
Substituindo o comportamento da classe pai.

A busca para no momento exato em que o método é encontrado na hierarquia inferior.



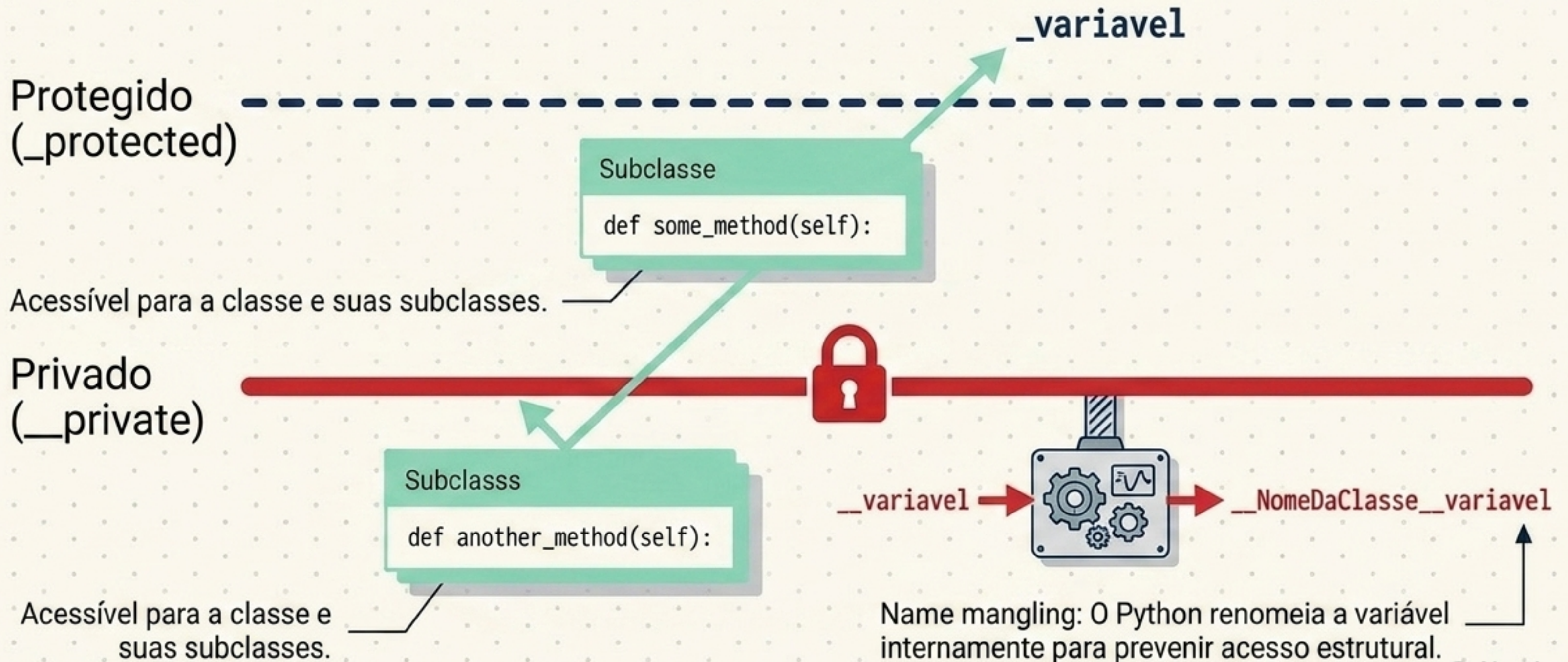
Estendendo Comportamentos: O Poder do super()

Aumentando métodos em vez de substituí-los.



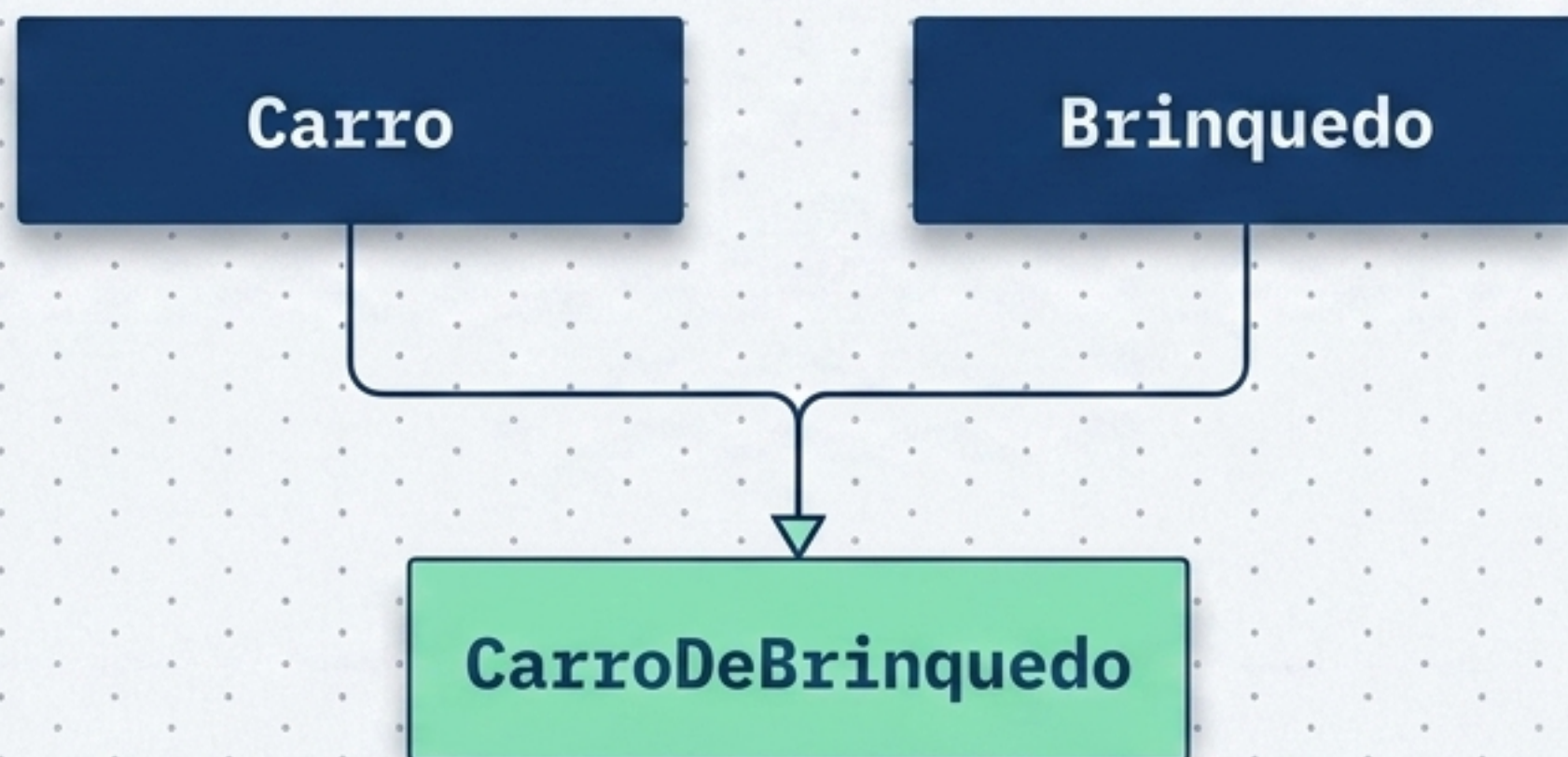
Encapsulamento e Convenções de Nomenclatura

Níveis de acesso estrutural em hierarquias.



Herança Múltipla: A Convergência

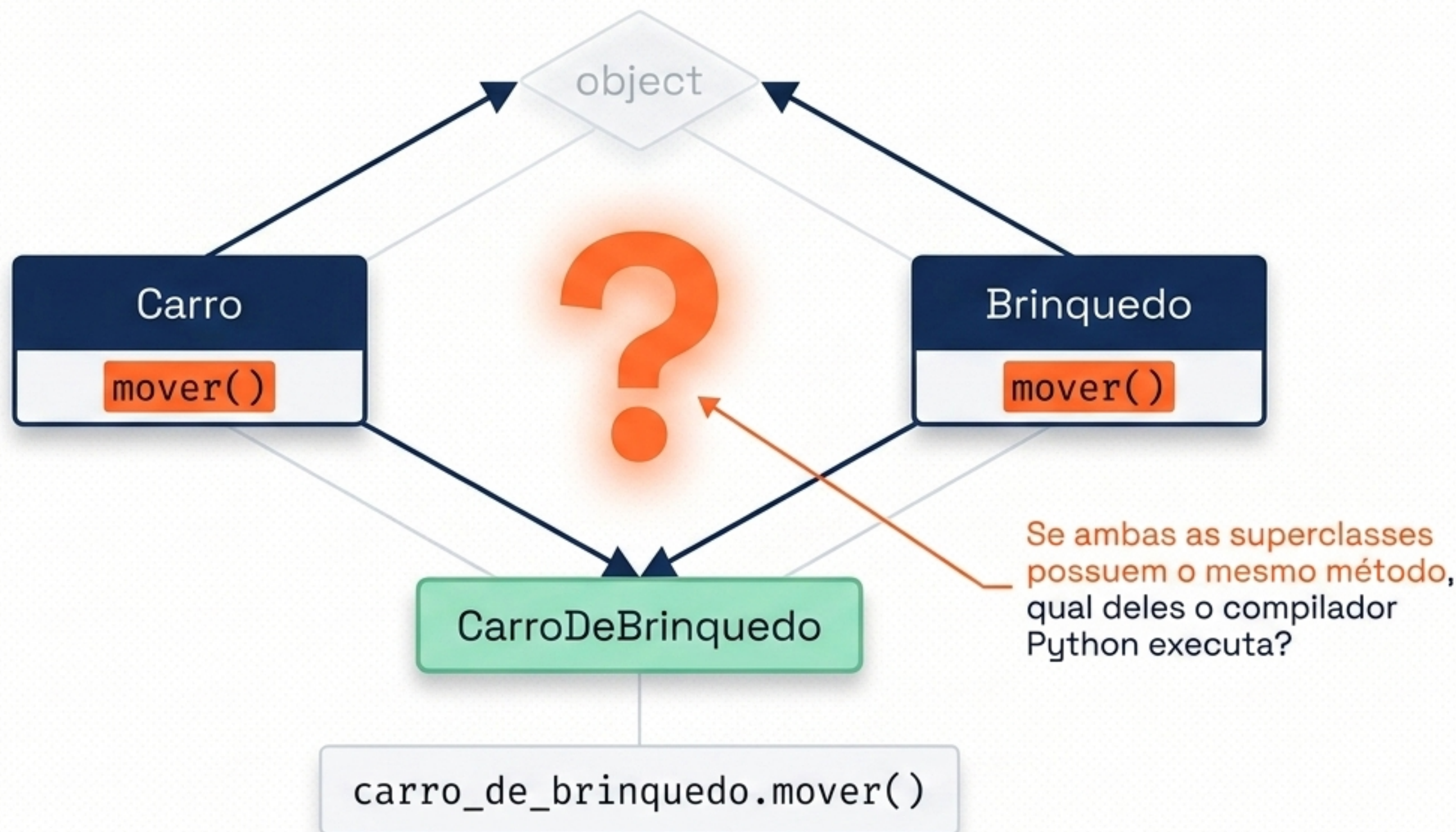
Python permite herdar de múltiplas superclasses simultaneamente, combinando todos os atributos.



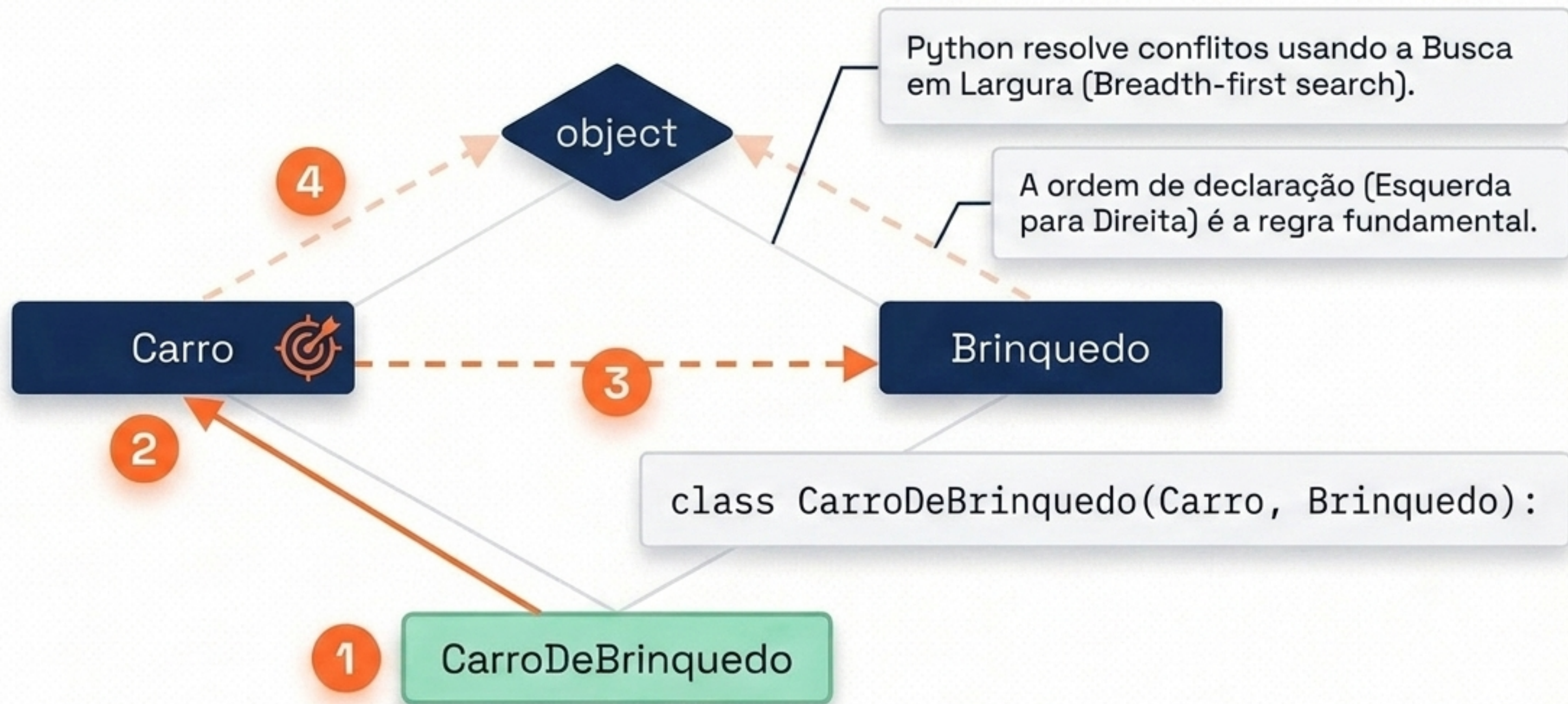
```
class CarroDeBrinquedo(Carro, Brinquedo):  
    pass
```


A Ameaça da Complexidade: O Problema do Diamante

O que acontece quando há ambiguidade na herança múltipla?



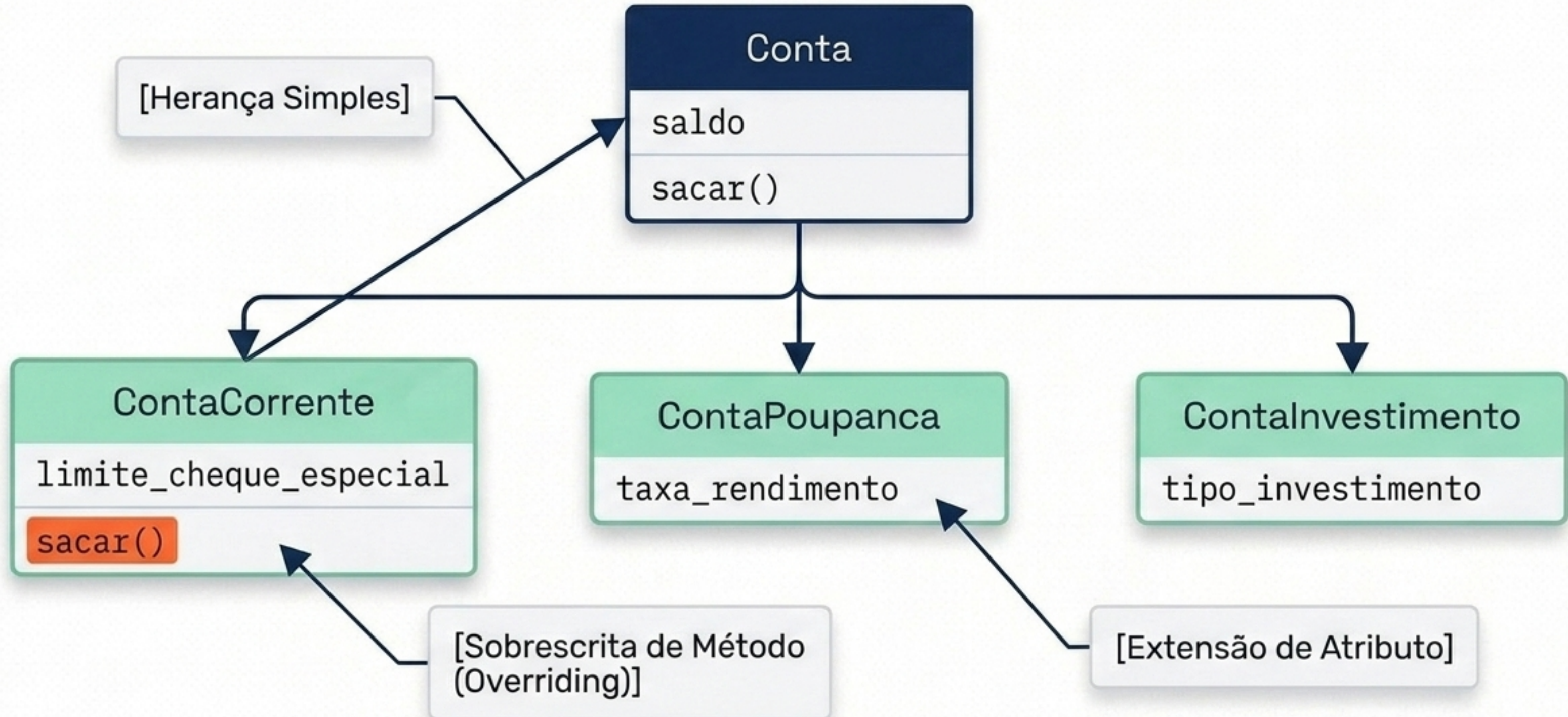
A Solução: Method Resolution Order (MRO)



TypeError: Hierarquias inconsistentes gerarão falha ao criar o MRO.

Síntese da Arquitetura: Ecossistema de Contas

Aplicando todos os conceitos em um cenário do mundo real.



Resumo da Arquitetura (Cheat Sheet)



Fundamento

Subclasses herdam todo o código da superclasse. Toda classe herda secretamente de object.



Diagnóstico

"É-um" determina Herança (subclassing).

"Faz-parte-de" determina Composição (atributos de instância).



Manipulação

Subclasses podem **adicionar métodos**, **sobrescrever lógicas antigas**, ou estender a classe pai usando `super()`.



Herança Múltipla

Permitida, porém governada rigidamente pelo **MRO** (**Busca em Largura**). A ordem de declaração (Esquerda -> Direita) define a prioridade.